

MediGuard AI: IoT-Based Medical Storage Monitoring and Cybersecurity Threat Detection System

IEEE-Style Technical Project Documentation

Prepared for Academic Project Submission

Version 2.0 | Generated Documentation

Abstract

MediGuard AI is an IoT-enabled and AI-assisted monitoring system designed to protect sensitive medical storage environments by supervising temperature, humidity, device behavior, and cybersecurity events. The system connects a real ESP32 and DHT11 sensor to a Flask web application through a sensor-reading API. A clinical dashboard provides medical/environmental monitoring, while a separate SOC dashboard supports cybersecurity visibility, vulnerability scoring, attack classification, isolation, and restoration. The project combines rule-based environmental thresholds with three AI models: a condition classification model, an anomaly detection model, and an attack decision model. This document presents the complete system design, workflow, AI methodology, cybersecurity matrix, dashboards, implementation details, model metrics, confusion matrices, and future improvements in an IEEE-style technical format.

Keywords—MediGuard AI, ESP32, DHT11, Flask, medical storage monitoring, anomaly detection, cybersecurity, SOC dashboard, vulnerability matrix, machine learning.

Table of Contents

- I. Introduction
- II. System Objectives and Scope
- III. System Architecture
- IV. System Flowchart
- V. Functional Requirements
- VI. AI Model Design
- VII. AI Model Evaluation and Comparison
- VIII. Cybersecurity Design and Vulnerability Matrix
- IX. Dashboard Design
- X. Implementation Summary
- XI. Testing Strategy
- XII. Limitations and Future Work
- XIII. Conclusion
- References
- Appendices

I. Introduction

Medical products such as vaccines, refrigerated medicines, and temperature-sensitive supplies require continuous monitoring to reduce the risk of spoilage, operational failure, and unsafe storage conditions. In modern connected environments, these monitoring systems are also exposed to cybersecurity risks because sensor data is transmitted through networks and stored or processed by software services.

MediGuard AI addresses both sides of the problem: environmental safety and cybersecurity awareness. The system accepts real temperature and humidity readings from an ESP32 sensor through a Flask API, classifies the environmental state, evaluates abnormal behavior, and displays separate outputs for clinical users and SOC users. This separation is important because a real physical temperature rise should be treated as an environmental warning or failure, not automatically as a cyber attack.

II. System Objectives and Scope

The objective of MediGuard AI is to build a practical prototype that demonstrates how medical storage monitoring can be enhanced using IoT, machine learning, and cybersecurity analysis. The scope focuses on a single real ESP32 room sensor connected over Wi-Fi, a Flask backend, role-based dashboards, and AI-based decision support.

- Monitor real temperature and humidity readings from an ESP32 + DHT11 sensor.
- Classify storage/environment state as Normal, Warning, or Critical.
- Distinguish environmental failure from cyber attack behavior.
- Provide a Doctor dashboard for clinical/environmental monitoring only.
- Provide a SOC dashboard for cybersecurity analysis, vulnerability matrix, attack details, isolation, and restoration.
- Maintain cyber attack state until SOC isolation/restoration action is performed.
- Support evaluation of three AI models with accuracy, precision, recall, F1-score, and confusion matrices.

III. System Architecture

The architecture follows a layered design. The sensor layer collects physical measurements. The backend layer receives and validates readings. The AI layer classifies condition, anomaly status, and attack/security state. The presentation layer separates clinical monitoring from cybersecurity operations.

Layer	Main Components	Responsibility
Sensor Layer	ESP32 + DHT11	Collect temperature and humidity readings from the monitored room.
Communication Layer	Wi-Fi + HTTP POST	Send sensor readings to the Flask API endpoint <code>/api/sensor_reading</code> .
Backend Layer	Flask app.py	Process readings, apply rules, run AI models, update CSV storage, and return API responses.
AI Layer	Condition, Anomaly, Attack Models	Predict environment condition, detect abnormal behavior, and classify cyber/environmental situation.
Clinical Layer	Doctor Dashboard	Display environmental condition, risk score, reason, and action without cybersecurity controls.
SOC Layer	SOC Dashboard	Display attack state, attack reason, vulnerability matrix, isolate/restore controls, and security alerts.
Storage Layer	devices.csv, logs.csv, model .pkl files	Store current sensor state, event logs, and trained machine learning models.

Table I. MediGuard AI layered system architecture.

IV. System Flowchart

The system workflow begins with sensor data input, continues through preprocessing and AI classification, then separates environmental safety decisions from cybersecurity decisions. The flowchart below represents the complete high-level logic used by the project.

MediGuard AI - System Flowchart

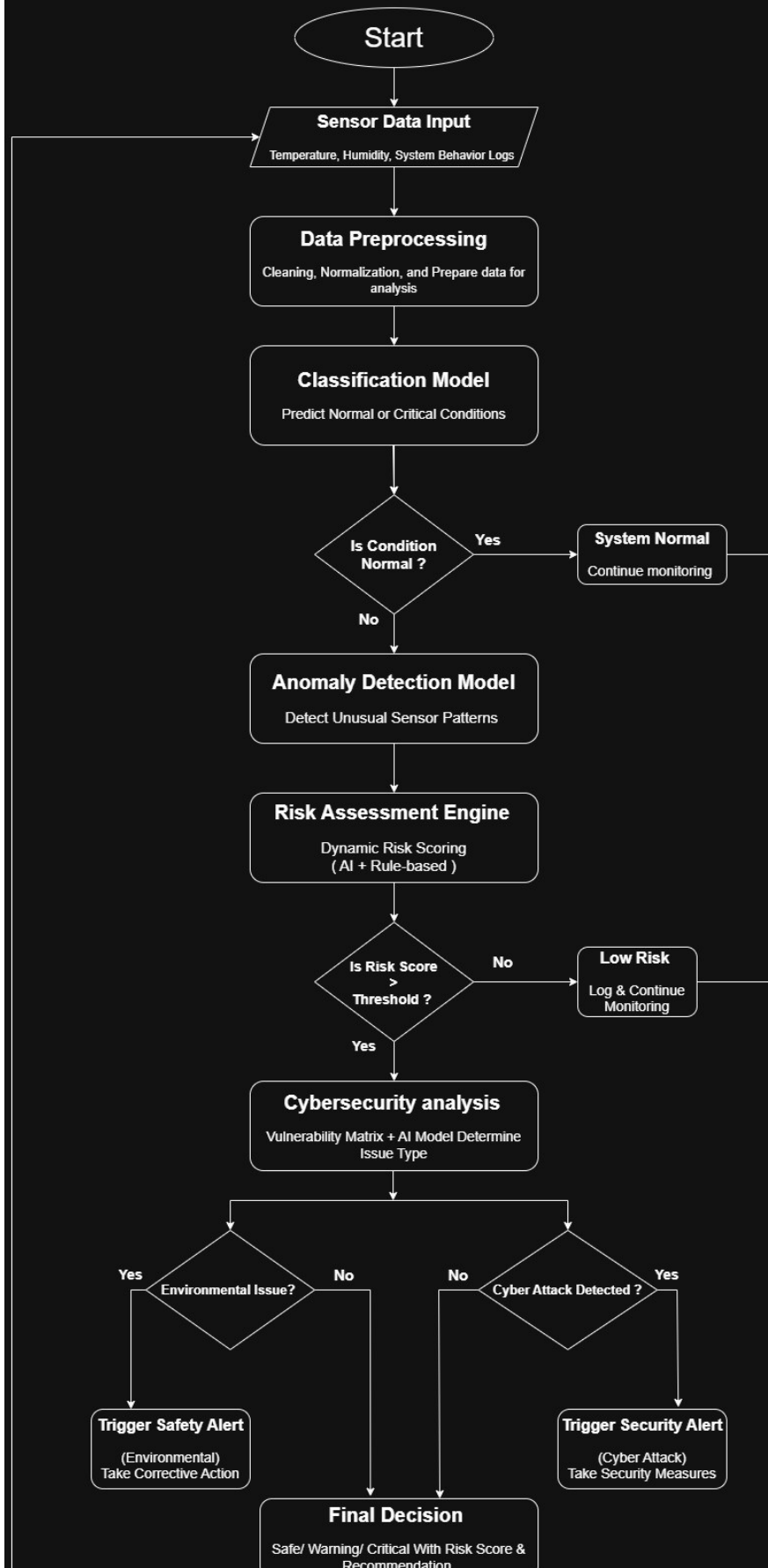


Fig. 1. MediGuard AI system flowchart showing sensor input, AI analysis, risk assessment, cybersecurity analysis, final decision, and logging.

V. Functional Requirements

ID	Requirement	Description
FR-01	Sensor Reading API	The system shall receive temperature and humidity readings from the ESP32 through /api/sensor_reading.
FR-02	Doctor Login	The system shall allow doctor login for monitoring-only access.
FR-03	SOC Login	The system shall allow SOC login for cybersecurity operations.
FR-04	Environmental Classification	The system shall classify room status as Normal, Warning, or Critical using threshold rules and AI support.
FR-05	Cyber Attack Persistence	When a cyber incident is detected or simulated, the attack status shall persist until SOC restore action.
FR-06	Sensor Isolation	The SOC user shall be able to isolate the sensor so new readings do not overwrite the security state.
FR-07	Sensor Restoration	The SOC user shall be able to restore the sensor after validation.
FR-08	Vulnerability Matrix	The SOC dashboard shall show a dynamic matrix with likelihood, impact, score, level, threat, and recommendation.
FR-09	Logging	The system shall save readings and decisions in logs.csv for audit and reporting.

Table II. Functional requirements of the MediGuard AI system.

VI. AI Model Design

The project uses three AI models aligned with the system logic. The condition model predicts whether sensor readings indicate normal/warning or critical conditions. The anomaly model learns normal and warning behavior patterns and identifies unusual patterns. The attack model supports final state classification using environmental risk, vulnerability score, anomaly flag, and critical prediction.

Model	Algorithm	Input Features	Output	Purpose
Condition Classification Model	Random Forest Classifier	Serial reading, temperature C/F, humidity, object temperature, cooling status	Normal/Warning or Critical	Predict physical condition severity from sensor readings.
Anomaly Detection Model	Isolation Forest	Safe and warning sensor behavior features	Normal Pattern or Anomalous	Detect unusual behavior such as abnormal patterns or manipulated-looking readings.
Attack Detection Model	Random Forest Classifier	Environmental risk, vulnerability score, anomaly flag, critical prediction	Normal Operation, Environmental Warning, Environmental Failure, Suspicious Behavior, Possible	Support final cybersecurity/environmental decision making.

			Cyber Attack	
--	--	--	--------------	--

Table III. AI model design summary.

The uploaded training script creates aligned synthetic training data, trains the three models, and saves them as condition_model.pkl, anomaly_model.pkl, attack_model.pkl, model.pkl, and feature_columns.pkl. The evaluation results in this documentation were generated from the same model logic with a fixed random seed to make the reported metrics reproducible. Because the data is synthetic and rule-aligned, the condition and attack models can show very high scores. Real-world deployment should validate the models using real sensor logs collected over time.

VII. AI Model Evaluation and Comparison

This section adds the missing model evaluation requested for the project: accuracy, precision, recall, F1-score, support, confusion matrices, and comparison across the three AI models. For the anomaly model, a labeled validation set was constructed with normal-pattern and abnormal-pattern examples because Isolation Forest is an unsupervised algorithm and the uploaded training script originally did not print supervised anomaly metrics.

Model	Algorithm	Accuracy	Precision	Recall	F1-score	Comment
Condition Classification Model	Random Forest	100.00%	1.000	1.000	1.000	Excellent on synthetic separable condition data.
Anomaly Detection Model	Isolation Forest	84.92%	0.861	0.849	0.848	Good detection of abnormal patterns; some false alarms remain.
Attack Detection Model	Random Forest	100.00%	1.000	1.000	1.000	Excellent on rule-generated attack decision labels.

Table IV. Comparison of the three AI models.

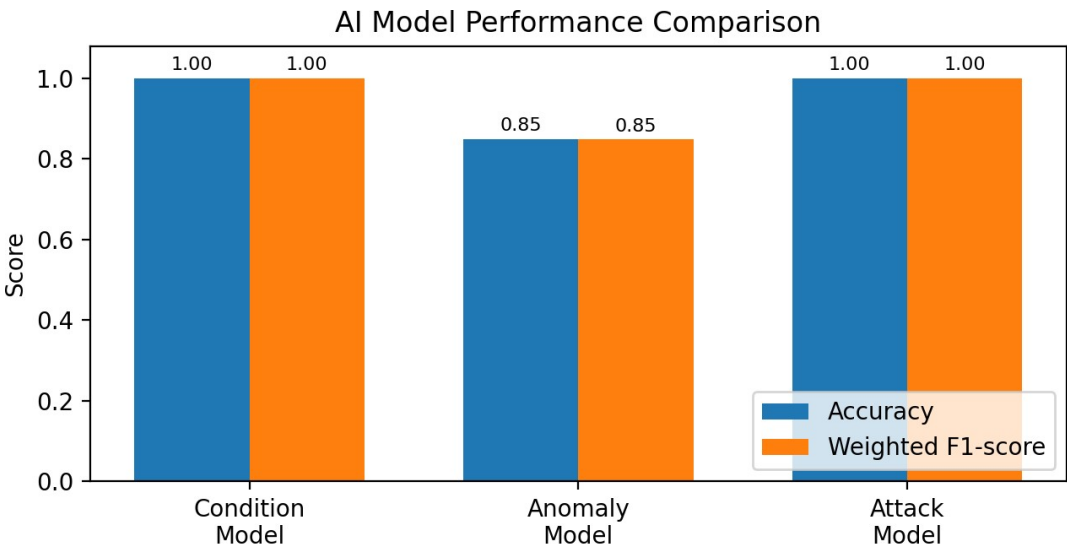


Fig. 2. Accuracy and weighted F1-score comparison of the three AI models.

A. Condition Classification Model Metrics

Class	Precision	Recall	F1-score	Support
-------	-----------	--------	----------	---------

Normal / Warning	1.000	1.000	1.000	700
Critical	1.000	1.000	1.000	400
Weighted Average	1.000	1.000	1.000	1100

Table V. Condition classification metrics.

Actual / Predicted	Normal/Warning	Critical
Normal/Warning	700	0
Critical	0	400

Table VI. Confusion matrix for the condition classification model.

B. Anomaly Detection Model Metrics

Class	Precision	Recall	F1-score	Support
Normal Pattern	0.925	0.760	0.834	600
Anomalous	0.796	0.938	0.862	600
Weighted Average	0.861	0.849	0.848	1200

Table VII. Anomaly detection metrics using a labeled validation set.

Actual / Predicted	Normal Pattern	Anomalous
Normal Pattern	456	144
Anomalous	37	563

Table VIII. Confusion matrix for the anomaly detection model.

C. Attack Detection Model Metrics

Class	Precision	Recall	F1-score	Support
Normal Operation	1.000	1.000	1.000	233
Environmental Warning	1.000	1.000	1.000	94
Environmental Failure	1.000	1.000	1.000	191
Suspicious Behavior	1.000	1.000	1.000	228
Possible Cyber Attack	1.000	1.000	1.000	54
Weighted Average	1.000	1.000	1.000	800

Table IX. Attack detection model classification metrics.

Actual / Predicted	Normal Operation	Environmental Warning	Environmental Failure	Suspicious Behavior	Possible Cyber Attack
Normal Operation	233	0	0	0	0
Environmental Warning	0	94	0	0	0
Environmental Failure	0	0	191	0	0
Suspicious Behavior	0	0	0	228	0
Possible Cyber Attack	0	0	0	0	54

Table X. Confusion matrix for the attack detection model.

VIII. Cybersecurity Design and Vulnerability Matrix

The SOC dashboard uses a vulnerability matrix to evaluate cybersecurity weaknesses. The matrix should be dynamic: the vulnerability rows remain the same, but likelihood, impact, score, and risk level change depending on the current attack type and isolation state. This avoids the unrealistic behavior where every attack makes every vulnerability critical.

Attack Type	Main Vulnerability Affected	Expected Matrix Behavior
Unauthorized Sensor Spoofing	No device authentication	Authentication-related risk becomes high or critical.
Man-in-the-Middle Modification	Unencrypted communication and no message integrity	Encryption and integrity risks become high or critical.
Replay Attack	No replay protection	Replay protection risk becomes high or critical.
DDoS / Flooding Attempt	No rate limiting	Rate-limiting risk becomes high or critical.
Malware Data Manipulation	No malware protection on gateway	Gateway malware risk becomes high or critical.
Impossible Sensor Payload	No input validation	Input validation risk becomes high or critical.

Table XI. Attack-specific vulnerability matrix behavior.

For the live demonstration, the recommended improvement is to require the real ESP32 to send a trusted sensor ID and secret token. A second laptop can then act as an attacker by sending fake requests to the Flask API without the correct token. The SOC dashboard should classify the event as Unauthorized Sensor Spoofing while preserving the real clinical sensor readings.

IX. Dashboard Design

The system uses role-based access to separate clinical monitoring from cybersecurity operations. This separation makes the project clearer and more realistic because doctors should not control cyber response actions, while SOC users should not confuse environmental heating with cyber compromise.

Dashboard	User	Main View	Allowed Actions
Doctor Dashboard	doctor / 123	Temperature, humidity, clinical status, environmental reason, risk score, recommended action	Monitoring only. No attack simulation, isolation, or restore controls.
SOC Dashboard	soc / 123	Security status, attack status, attack type, attack reason, vulnerability matrix, sensor isolation state	Simulate/demo cyber attack, isolate sensor, restore sensor, review vulnerability risk.

Table XII. Role-based dashboard design.

X. Implementation Summary

The backend is implemented using Flask. It loads trained AI models using joblib, stores sensor/device state in CSV format, logs events to logs.csv, and exposes routes for login, dashboards, sensor readings, logs, simulation, isolation, and restoration. The current system uses one real sensor named Real ESP32 Room Sensor. Real readings update temperature, humidity, clinical status, environmental reason, risk level, and action. Cyber attack state persists until SOC restoration.

File / Component	Role in Project
app.py	Main Flask backend, route handling, device state management, AI model calls, risk calculation, and SOC actions.
train_model.py	Generates aligned training data, trains the condition model, anomaly model, and attack model, and saves model files.
clinical_dashboard.html	Doctor interface for monitoring medical/environmental condition.
soc_dashboard.html	SOC interface for cybersecurity monitoring, attack details, vulnerability matrix, isolate and restore.
devices.csv	Stores the latest state of the real ESP32 room sensor.
logs.csv	Stores historical readings and decisions.
condition_model.pkl	Trained model for condition classification.
anomaly_model.pkl	Trained model for anomaly detection.
attack_model.pkl	Trained model for attack/environmental decision support.

Table XIII. Main implementation components.

XI. Testing Strategy

The testing strategy covers normal environmental readings, warning readings, critical readings, cybersecurity attack attempts, persistence behavior, and SOC response actions.

Test Case	Input / Action	Expected Result
Normal Reading	ESP32 sends 27 C and 50% humidity	Doctor dashboard shows Normal; SOC dashboard remains Secure.
Environmental Warning	ESP32 sends temperature between 30 and 37.9 C or humidity 20-29 / 71-80	Doctor dashboard shows Warning; SOC should not automatically show cyber attack.
Environmental Failure	ESP32 sends 38 C+ or humidity below 20 / above 80	Doctor dashboard shows Critical; SOC should classify as Environmental Failure, not cyber attack.
Unauthorized Laptop Request	Second laptop sends fake API request without trusted token	SOC dashboard shows Possible Cyber Attack and Unauthorized Sensor Spoofing.
Attack Persistence	Cyber attack detected, then ESP32 sends normal reading	Cyber attack remains active until SOC restore.
Isolation	SOC presses Isolate	Sensor state becomes Isolated and readings

		cannot overwrite SOC state.
Restore	SOC presses Restore	Attack state clears and sensor returns to active monitoring.

Table XIV. Recommended system test cases.

XII. Limitations and Future Work

The current version is a strong prototype, but it still has limitations that should be addressed before real deployment. Most model data is synthetic and aligned with project rules. The cybersecurity attack scenarios are prototype-level and should be expanded with stronger authentication, replay protection, secure transport, and real network monitoring.

- Collect a larger real dataset from the ESP32 sensor under normal, warning, and critical room conditions.
- Replace plain HTTP with HTTPS or MQTT over TLS for secure communication.
- Add device authentication using API tokens or HMAC signatures.
- Add timestamp and nonce validation to detect replay attacks more accurately.
- Add rate limiting to reduce flooding/DDoS risk against the Flask API.
- Store logs in a database instead of CSV for better reporting and audit integrity.
- Deploy the Flask application behind a production-grade server and firewall.
- Evaluate AI models on real collected sensor data and compare with synthetic-data performance.

XIII. Conclusion

MediGuard AI demonstrates a practical integration of IoT monitoring, AI-based decision support, and cybersecurity analysis for medical storage environments. The project separates environmental failures from cyber attacks, uses role-based dashboards for Doctor and SOC users, and evaluates three AI models with detailed performance metrics. The system is suitable as an academic prototype and can be extended into a more production-ready monitoring platform through stronger authentication, real datasets, secure communication, and enhanced SOC analytics.

References

- [1] MediGuard AI project source code, app.py, uploaded project file.
- [2] MediGuard AI training script, train_model.py, uploaded project file.
- [3] MediGuard AI initial project documentation, uploaded PDF file.
- [4] Scikit-learn documentation, RandomForestClassifier and IsolationForest APIs.
- [5] Flask documentation, route handling, sessions, and web application structure.
- [6] Espressif Systems, ESP32 documentation and Wi-Fi connectivity reference.
- [7] DHT11 temperature and humidity sensor module technical references.

Appendix A. Environmental Threshold Rules

Condition	Temperature Rule	Humidity Rule	Action
Normal	20-30 C	30-70%	Continue Monitoring
Warning	30-37.9 C	20-29% or 71-80%	Check Room Conditions
Critical	38 C or higher	Below 20% or above 80%	Immediate Environmental Check Required

Table XV. Environmental threshold rules used by MediGuard AI.

Appendix B. Suggested Final Demo Scenario

- Start Flask backend and open Doctor and SOC dashboards.
- Allow ESP32 to send normal room data; verify Normal and Secure states.
- Heat the DHT11 sensor; verify Environmental Warning or Environmental Failure, not cyber attack.
- Use a second laptop to send an unauthorized fake API request; verify Possible Cyber Attack on SOC dashboard.
- Show the dynamic vulnerability matrix and attack reason.
- Press Isolate and explain that the SOC stops trusting the sensor until validation.
- Press Restore and verify the system returns to normal monitoring.